

코드 리뷰 보고서

리뷰번호	001	작성자	정제은	작성날짜	2026.06.01
관련규칙	03. 민감한 가변적 클래스에 수정이 불가능한 래퍼를 제공하라 41. 배열이나 컬렉션을 반환하는 메서드에서 널 값 대신 빈 배열이나 컬렉션을 반환하라				
중요도	상				
대상코드	Before				
	<pre>package model; import java.io.Serializable; public class CardData implements Serializable { private static final long serialVersionUID = 1L; private PaymentInfo paymentInfo; // 결제 정보 private byte[] orderInfoHash; // 주문 정보 해시 private byte[] digitalSignature; // 인증서명 public CardData(PaymentInfo paymentInfo, byte[] orderInfoHash, byte[] digitalSignature) { this.paymentInfo = paymentInfo; this.orderInfoHash = orderInfoHash; this.digitalSignature = digitalSignature; } public PaymentInfo getPaymentInfo() { return paymentInfo; } public byte[] getOrderInfoHash() { return orderInfoHash; } public byte[] getDigitalSignature() { return digitalSignature; } }</pre>				
	After				
	<pre>package model; import java.io.Serializable; public class CardData implements Serializable { private static final long serialVersionUID = 1L; private PaymentInfo paymentInfo; // 결제 정보 private byte[] orderInfoHash; // 주문 정보 해시 private byte[] digitalSignature; // 인증서명 private byte[] paymentInfoSalt; public CardData(PaymentInfo paymentInfo, byte[] orderInfoHash, byte[] digitalSignature, byte[] paymentInfoSalt) { this.paymentInfo = paymentInfo; this.orderInfoHash = orderInfoHash != null ? orderInfoHash.clone() : null; this.digitalSignature = digitalSignature != null ? digitalSignature.clone() : null; this.paymentInfoSalt = paymentInfoSalt != null ? paymentInfoSalt.clone() : null; } public PaymentInfo getPaymentInfo() { return paymentInfo; } public byte[] getOrderInfoHash() { return orderInfoHash != null ? orderInfoHash.clone() : new byte[0]; } public byte[] getDigitalSignature() { return digitalSignature != null ? digitalSignature.clone() : new byte[0]; } public byte[] getPaymentInfoSalt() { return paymentInfoSalt != null ? paymentInfoSalt.clone() : new byte[0]; } }</pre>				
보안약점/ 보안취약점 이유 및 해결법	배열은 가변 객체이기 때문에 원본 배열 그대로 반환하면 외부에서 원본 배열을 임의로 수정할 수 있기에 보안에 취약하다(03). 그래서 생성자와 getter 메서드에서 clone()을 사용하여 복사본을 넘겨주는 방어적 복사로 수정했다. 그리고 원래 new byte[0]가 아닌 null을 반환했는데, 41번 규칙을 적용해 null이 아닌 빈 배열을 반환하도록 수정했다. (model 패키지 속 파일들을 이렇게 수정하였음)				

코드 리뷰 보고서

리뷰번호	002	작성자	정제은	작성날짜	2026.06.01
관련규칙	06. 출력을 적절하게 인코딩하거나 이스케이핑하라.				
중요도	중				
대상코드	Before				
	<pre>package model; import java.io.Serializable; public class PaymentInfo implements Serializable { private static final long serialVersionUID = 1L; private String cardNumber; private int price; public PaymentInfo(String cardNumber, int price) { this.cardNumber = cardNumber; this.price = price; } public String getCardNumber() { return cardNumber; } public int getPrice() { return price; } public String toString() { String payment = "카드번호: " + cardNumber + ", 결제금액: " + price; return payment; } }</pre>				
	After				
	<pre>✓ public class PaymentInfo implements Serializable { private static final long serialVersionUID = 1L; private char[] cardNumber; private int price; public PaymentInfo(char[] cardNumber, int price) { this.cardNumber = cardNumber != null ? cardNumber.clone() : null; this.price = price; } public char[] getCardNumber() { return cardNumber != null ? cardNumber.clone() : new char[0]; } public int getPrice() { return price; } // 카드번호 즉시 지우기 public void clearCardNumber() { if (this.cardNumber != null) { Arrays.fill(this.cardNumber, '0'); // 메모리 공간을 문자 '0'으로 덮어씀 } } public String toString() { if (cardNumber == null cardNumber.length == 0) { return "카드번호: *****"; } StringBuilder masked = new StringBuilder(); for (int i = 0; i < cardNumber.length; i++) { if (i < 4) { masked.append(cardNumber[i]); // 앞 4자리만 평문 노출 } else { if (cardNumber[i] == '-') { masked.append('-'); // 포맷을 대시 기호는 유지 } else { masked.append('*'); // 나머지 금융 데이터는 전부 마스킹 } } } return masked.toString(); } }</pre>				
보안약점/ 보안취약점 이유 및 해결법	수정전 사용자의 카드 번호가 그대로 결합해 콘솔 스트림으로 출력되는 문제가 있음. 출력 스트림으로 민감정보가 새어 나가지 않도록 마스킹 이스케이프 처리를 거친 후 안전하게 텍스트를 출력하도록 수정함.				

코드 리뷰 보고서

리뷰번호	003	작성자	정제은	작성날짜	2026.06.01
관련규칙	43. 닫을 수 있는 자원을 안전하게 처리하기 위해서는 자원동반-try 문장을 사용하라				
중요도	중				
대상코드	Before				
	<pre>// 배달의 민족에서 보낸 2차 전자봉투 개봉 public BaeminToCardData openEnvelope(DigitalEnvelope cardEnvelope) throws InvalidKeyException, NoSuchAlgorithmException, NoSuchPaddingException, Illegal byte[] decrypted = EnvelopeUtil.openEnvelope(cardEnvelope, privateKey); BaeminToCardData cardData = (BaeminToCardData) bytesToObject(decrypted); return cardData; }</pre>				
	<pre>66 } 67 68 // 객체 직렬화 69 public byte[] objectToBytes(Object obj) throws IOException { 70 ByteArrayOutputStream bos = new ByteArrayOutputStream(); 71 ObjectOutputStream oos = new ObjectOutputStream(bos); 72 73 oos.writeObject(obj); 74 oos.flush(); 75 76 byte[] bytes = bos.toByteArray(); 77 return bytes; 78 } 79 } 80</pre>				
	After				
	<pre>// 배달의 민족에서 보낸 2차 전자봉투 개봉 public BaeminToCardData openEnvelope(DigitalEnvelope cardEnvelope) throws InvalidKeyException, NoSuchAlgorithmException, NoSuchPaddingException, Illegal byte[] decrypted = EnvelopeUtil.openEnvelope(cardEnvelope, privateKey); return (BaeminToCardData) SerializeUtil.bytesToObject(decrypted); } package crypto; import java.io.ByteArrayInputStream; import java.io.ByteArrayOutputStream; import java.io.IOException; import java.io.ObjectInputStream; import java.io.ObjectOutputStream; import java.io.ObjectOutputSteam; public class SerializeUtil { // 객체 직렬화 (Object -> byte[]) public static byte[] objectToBytes(Object obj) throws IOException { try (ByteArrayOutputStream bos = new ByteArrayOutputStream(); ObjectOutputStream oos = new ObjectOutputStream(bos)) { oos.writeObject(obj); oos.flush(); return bos.toByteArray(); } } // 객체 역직렬화 (byte[] -> Object) public static Object bytesToObject(byte[] data) throws IOException, ClassNotFoundException { try (ByteArrayInputStream bis = new ByteArrayInputStream(data); ObjectInputStream ois = new ObjectInputStream(bis)) { return ois.readObject(); } } }</pre>				
보안약점/ 보안취약점 이유 및 해결법	entity패키지 속 파일들 아래엔 객체 직렬화와 역직렬화를 위한 코드가 공통으로 들어가 있었다. 코드의 가독성을 위해 crypto 패키지에 SerializeUtil 클래스를 만들었고, 기존의 코드에선 스트림이 close되지 않은 채 데이터만 리턴되어 try문장을 사용하여 직렬화와 역직렬화 코드를 수정했다, 여기에 맞추어 entity 패키지 속 파일들을 수정했다.				

코드 리뷰 보고서

리뷰번호	004	작성자	정제은	작성날짜	2026.06.02
관련규칙	12. 안전하지 않거나 약한 암호 알고리즘을 사용하지 말라 26. 메서드의 결과값에 대해 항상 피드백을 제공하라 42. 예외적인 상황에 대해서만 예외를 사용하라				
중요도	상				
대상코드	Before				
	<pre>// 대칭 키 암호화 public static byte[] encrypt(byte[] data, SecretKey secretKey) throws NoSuchAlgorithmException, NoSuchPaddingException, InvalidKeyException, IllegalBlockSizeException, BadPaddingException { Cipher cipher = Cipher.getInstance("AES"); cipher.init(Cipher.ENCRYPT_MODE, secretKey); byte[] encrypted = cipher.doFinal(data); return encrypted; }</pre> <pre>// 대칭 키 복호화 public static byte[] decrypt(byte[] encryptedData, SecretKey secretKey) throws NoSuchAlgorithmException, NoSuchPaddingException, InvalidKeyException, IllegalBlockSizeException, BadPaddingException { Cipher cipher = Cipher.getInstance("AES"); cipher.init(Cipher.DECRYPT_MODE, secretKey); byte[] decrypted = cipher.doFinal(encryptedData); return decrypted; }</pre>				
	After				
	<pre>public class AESUtil { // 대칭 키 생성 public static SecretKey generateKey() throws NoSuchAlgorithmException { KeyGenerator keyGen = KeyGenerator.getInstance("AES"); keyGen.init(128); SecretKey secretKey = keyGen.generateKey(); return secretKey; } // 대칭 키 암호화 (IV를 생성해서 암호문 앞에 붙여서 반환) public static byte[] encrypt(byte[] data, SecretKey secretKey) throws NoSuchAlgorithmException, NoSuchPaddingException, InvalidKeyException, InvalidAlgorithmParameterException, IllegalBlockSizeException, BadPaddingException { // ALGORITHM("AES/CBC/PKCS5Padding") 사용 Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding"); // CBC 모드에서 사용할 IV 생성 byte[] iv = new byte[16]; // 16바이트 크기의 난수 바이트 배열 생성 SecureRandom random = new SecureRandom(); random.nextBytes(iv); // 무작위 바이트 배열 생성 IvParameterSpec ivSpec = new IvParameterSpec(iv); cipher.init(Cipher.ENCRYPT_MODE, secretKey, ivSpec); byte[] encrypted = cipher.doFinal(data); // 복호화할 때 IV가 필요하므로 [iv + 암호문] 형태로 결합하여 반환 byte[] combined = new byte[iv.length + encrypted.length]; System.arraycopy(iv, 0, combined, 0, iv.length); System.arraycopy(encrypted, 0, combined, iv.length, encrypted.length); return combined; } // 대칭 키 복호화 public static byte[] decrypt(byte[] combinedData, SecretKey secretKey) throws NoSuchAlgorithmException, NoSuchPaddingException, InvalidKeyException, InvalidAlgorithmParameterException, IllegalBlockSizeException, BadPaddingException { // 암호문 앞부분에서 IV 추출 byte[] iv = new byte[16]; System.arraycopy(combinedData, 0, iv, 0, iv.length); IvParameterSpec ivSpec = new IvParameterSpec(iv); // 실제 암호문 데이터 추출 int encryptedSize = combinedData.length - 16; byte[] encrypted = new byte[encryptedSize]; System.arraycopy(combinedData, 16, encrypted, 0, encryptedSize); // ALGORITHM("AES/CBC/PKCS5Padding") 사용 Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding"); cipher.init(Cipher.DECRYPT_MODE, secretKey, ivSpec); return cipher.doFinal(encrypted); } }</pre>				
보안약점/ 보안취약점 이유 및 해결법	12) 암호화 메서드에서, Cipher.getInstance에서 AES만 사용하면 패턴 노출이 되어 통계적 분석 공격에 노출이 될 수 있다. 그래서 CBC모드와 IV(난수)를 사용해서 통계적 분석 공격을 막았다. (AES/CBC/PKCS5Padding으로 변경하기 위해 iv를 생성함.) 26, 42) ‘decrypt’ 메서드 시그니처에 ‘InvalidAlgorithmParameterException’을 명시하여, 전송 중에 발생한 IV 오염이나 잘못된 알고리즘 파라미터 주입 공격 시 결함 흐름을 시스템이 정상적으로 통제하고 상위 가용 영역으로 전파하도록 설계함.				

코드 리뷰 보고서

리뷰번호	005	작성자	정제은	작성날짜	2026.06.02
관련규칙	01. 민감-데이터의 수명을 제한하라 22. 변수들의 영역 범위를 최소화하라				
중요도	상				
대상코드	Before				
	<pre>public class EnvelopeUtil { // 전자봉투 생성 public static DigitalEnvelope sealEnvelope(byte[] data, PublicKey publicKey) throws NoSuchAlgorithmException, InvalidKeyException, NoSuchPaddingException, IllegalBlockSizeException, BadPaddingException { // AES 키 생성 SecretKey aesKey = AESUtil.generateKey(); // 골재 정보 AES 암호화 byte[] encryptedData = AESUtil.encrypt(data, aesKey); // AES 키를 공개 키로 암호화 byte[] aesKeyBytes = aesKey.getEncoded(); byte[] encryptedKey = RSAUtil.encrypt(aesKeyBytes, publicKey); // 전자봉투 생성 DigitalEnvelope digitalEnvelope = new DigitalEnvelope(encryptedData, encryptedKey); return digitalEnvelope; } // 전자봉투 개봉 public static byte[] openEnvelope(DigitalEnvelope envelope, PrivateKey privateKey) throws InvalidKeyException, NoSuchAlgorithmException, NoSuchPaddingException, IllegalBlockSizeException, BadPaddingException { // 골재 키로 암호화된 AES 키를 개인 키로 복호화 byte[] aesKeyBytes = RSAUtil.decrypt(envelope.getEncryptedKey(), privateKey); // byte[] 형태의 키를 SecretKey 타입으로 복원 SecretKey aesKey = new SecretKeySpec(aesKeyBytes, "AES"); // (B01) 복호화 byte[] data = AESUtil.decrypt(envelope.getEncryptedData(), aesKey); return data; } }</pre>				
	After				
	<pre>public class EnvelopeUtil { // 전자봉투 생성 public static DigitalEnvelope sealEnvelope(byte[] data, PublicKey publicKey) throws NoSuchAlgorithmException, InvalidKeyException, NoSuchPaddingException, IllegalBlockSizeException, BadPaddingException, InvalidAlgorithmParameterException { // AES 키 생성 SecretKey aesKey = AESUtil.generateKey(); // 골재 정보 AES 암호화 byte[] encryptedData = AESUtil.encrypt(data, aesKey); // AES 키를 RSA 공개 키로 암호화 byte[] aesKeyBytes = aesKey.getEncoded(); byte[] encryptedKey; try { encryptedKey = RSAUtil.encrypt(aesKeyBytes, publicKey); } finally { // 메모리에 남아있는 AES 키 바이트 배열 제거 Arrays.fill(aesKeyBytes, (byte) 0); } // 전자봉투 생성 후 반환 return new DigitalEnvelope(encryptedData, encryptedKey); } // 전자봉투 개봉 public static byte[] openEnvelope(DigitalEnvelope envelope, PrivateKey privateKey) throws InvalidKeyException, NoSuchAlgorithmException, NoSuchPaddingException, IllegalBlockSizeException, BadPaddingException, InvalidAlgorithmParameterException { // RSA 공개 키로 암호화된 AES 키를 RSA 개인 키로 복호화 byte[] aesKeyBytes = RSAUtil.decrypt(envelope.getEncryptedKey(), privateKey); SecretKey aesKey; try { // byte[] 형태의 키를 SecretKey 타입으로 복원 aesKey = new SecretKeySpec(aesKeyBytes, "AES"); // 데이터 복호화 후 반환 return AESUtil.decrypt(envelope.getEncryptedData(), aesKey); } finally { // 복호화 직후 메모리에 남은 평문 바이트 배열 제거 Arrays.fill(aesKeyBytes, (byte) 0); } } }</pre>				
보안약점/ 보안취약점 이유 및 해결법	AES키를 공개키로 암호화할 때, 민감 데이터는 GC가 수거하기 전까지 메모리에 평문으로 남아 있다. 이를 해결하기 위해 Arrays.fill(array, byte(0))기법으로 0으로 초기화하여 파기하도록 수정했다. 그리고, 전자봉투 생성하는 코드에서 불필요한 로컬 변수를 지우고 인스턴스를 생성하며 즉시 반환하도록 변경하였다.(22번)				

코드 리뷰 보고서

리뷰번호	006	작성자	정제은	작성날짜	2026.06.02
관련규칙	12. 안전하지 않거나 약한 암호 알고리즘을 사용하지 말라 13. 해시함수를 이용하여 패스워드를 저장하라 14. SecureRandom에 적절한 시드를 제공하도록 보장하라 22. 변수들의 영역 범위를 최소화하라				
중요도	상				
대상코드	Before				
	<pre> package crypto; import java.security.MessageDigest; import java.security.NoSuchAlgorithmException; ✓ public class HashUtil { // SHA-256 해시 생성 ✓ public static byte[] hash(byte[] data) throws NoSuchAlgorithmException { MessageDigest md = MessageDigest.getInstance("SHA-256"); md.update(data); byte[] mdSHA256 = md.digest(); return mdSHA256; } // byte[]를 hex 문자열로 변환 ✓ public static String bytesToHex(byte[] bytes) { StringBuffer sb = new StringBuffer(); for (byte b : bytes) { sb.append(String.format("%02x", b)); } return sb.toString(); } // 두 개의 해시 결합 ✓ public static byte[] combineHash(byte[] hash1, byte[] hash2) { byte[] combined = new byte[hash1.length + hash2.length]; System.arraycopy(hash1, 0, combined, 0, hash1.length); System.arraycopy(hash2, 0, combined, hash1.length, hash2.length); return combined; } } </pre>				
대상코드	After				
	<pre> public class HashUtil { // 솔트(Salt) 생성 public static byte[] getSalt() { byte[] salt = new byte[16]; SecureRandom random = new SecureRandom(); random.nextBytes(salt); return salt; } // SHA-256 해시 생성 (솔트 포함) public static byte[] hashSalt(byte[] data, byte[] salt) throws NoSuchAlgorithmException { MessageDigest md = MessageDigest.getInstance("SHA-256"); // 솔트 + 데이터 md.update(salt); md.update(data); return md.digest(); // 변수 선언 없이 즉시 반환 } public static byte[] hash(byte[] data) throws NoSuchAlgorithmException { MessageDigest md = MessageDigest.getInstance("SHA-256"); md.update(data); return md.digest(); } // byte[]를 hex 문자열로 변환 public static String bytesToHex(byte[] bytes) { if (bytes == null) { return ""; } StringBuilder sb = new StringBuilder(bytes.length * 2); for (int i = 0; i < bytes.length; i++) { sb.append(String.format("%02x", bytes[i])); } return sb.toString(); } // 두 개의 해시 결합 public static byte[] combineHash(byte[] hash1, byte[] hash2) { byte[] combined = new byte[hash1.length + hash2.length]; System.arraycopy(hash1, 0, combined, 0, hash1.length); System.arraycopy(hash2, 0, combined, hash1.length, hash2.length); return combined; } } </pre>				
보안약점/ 보안취약점 이유 및 해결법	12번) 해시 생성 시 salt를 먼저 주입하여 레인보우 테이블 공격을 방어하였다. 13번) hash()가 카드 번호를 검증할 때 사용되는데, SHA-256 해시값이면 디코딩될 수 있기에 salt를 사용했다. (해시를 생성할 때마다 사용자마다 고유하고 무작위로 생성되는 고강도 데이터인 salt를 결합하여 함께 해싱했다.) 14번) salt값은 사용자마다, 생성할 때마다 중복되거나 예측되어서는 안 됨으로 RecureRandom을 사용하여 salt값을 생성하도록 수정하였다. 22번) 불필요한 임시 변수 선언 줄이고, 연산 결과를 즉시 반환함으로써 생명주기 영역 최소화, 그리고 StringBuffer 대신 StringBuilder를 사용하여 과도한 코드와 오버헤드를 제거하였다.				

코드 리뷰 보고서

리뷰번호	007	작성자	정제은	작성날짜	2026.06.02
관련규칙	12. 안전하지 않거나 약한 암호 알고리즘을 사용하지 말라 22. 변수들의 영역 범위를 최소화하라 43. 닫을 수 있는 자원을 안전하게 처리하기 위해서는 자원동반-try문장을 사용하라				
중요도	상				
대상코드	Before				
	<pre>public class RSATUtil { // 비대칭 키 생성 public static KeyPair generateKeyPair() throws NoSuchAlgorithmException { KeyPairGenerator keyPairGen = KeyPairGenerator.getInstance("RSA"); keyPairGen.initialize(1024); KeyPair keyPair = keyPairGen.generateKeyPair(); return keyPair; } // 공개 키 저장 public static void savePublicKey(String fname, PublicKey publicKey) throws FileNotFoundException, IOException { File directory = new File("keys"); if (!directory.exists()) { directory.mkdirs(); } File file = new File(directory, fname); try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(file))) { oos.writeObject(publicKey); } } }</pre>				
대상코드	After				
	<pre>public class RSATUtil { private static final String TRANSFORMATION = "RSA/ECB/OAEPWithMD5AndSHA1Padding"; private static final int KEY_SIZE = 2048; // 비대칭 키 생성 public static KeyPair generateKeyPair() throws NoSuchAlgorithmException { KeyPairGenerator keyPairGen = KeyPairGenerator.getInstance("RSA"); keyPairGen.initialize(KEY_SIZE); return keyPairGen.generateKeyPair(); // 변수 선언 없이 즉시 반환 } // 공개 키 암호화 public static byte[] encrypt(byte[] data, PublicKey publicKey) throws NoSuchAlgorithmException, NoSuchPaddingException, InvalidKeyException, IllegalBlockSizeException, BadPaddingException { Cipher cipher = Cipher.getInstance(TRANSFORMATION); cipher.init(Cipher.ENCRYPT_MODE, publicKey); return cipher.doFinal(data); } // 개인 키 복호화 public static byte[] decrypt(byte[] encryptedData, PrivateKey privateKey) throws NoSuchAlgorithmException, NoSuchPaddingException, InvalidKeyException, IllegalBlockSizeException, BadPaddingException { Cipher cipher = Cipher.getInstance(TRANSFORMATION); cipher.init(Cipher.DECRYPT_MODE, privateKey); return cipher.doFinal(encryptedData); } // 공개 키 저장 public static void savePublicKey(String fname, PublicKey publicKey) throws IOException { File directory = new File("keys"); if (!directory.exists()) { directory.mkdirs(); } File file = new File(directory, fname); try (FileOutputStream fos = new FileOutputStream(file); ObjectOutputStream oos = new ObjectOutputStream(fos)) { oos.writeObject(publicKey); } } // 개인 키 저장 public static void savePrivateKey(String fname, PrivateKey privateKey) throws IOException { File directory = new File("keys"); if (!directory.exists()) { directory.mkdirs(); } File file = new File(directory, fname); try (FileOutputStream fos = new FileOutputStream(file); ObjectOutputStream oos = new ObjectOutputStream(fos)) { oos.writeObject(privateKey); } } // 공개 키 불러오기 public static PublicKey loadPublicKey(String fname) throws FileNotFoundException, IOException, ClassNotFoundException { File directory = new File("keys"); File file = new File(directory, fname); try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(file))) { PublicKey publicKey = (PublicKey) ois.readObject(); return publicKey; } } // 개인 키 불러오기 public static PrivateKey loadPrivateKey(String fname) throws FileNotFoundException, IOException, ClassNotFoundException { File directory = new File("keys"); File file = new File(directory, fname); try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(file))) { PrivateKey privateKey = (PrivateKey) ois.readObject(); return privateKey; } } }</pre>				
보안약점/ 보안취약점 이유 및 해결법	12번) 패딩 모드를 OAEP로 명시하여 고강도 보안 확보, RSA 키 길이를 1024가 취약해서 2048로 변경. 22번)중간 변수 없이 즉시 반환함. 43번) 공개키와 개인키 파일을 디스크에 입출력과정에서 파일 락현상이나 메모리 누수 발생해서 try-with-recources 구문을 추가함.				

코드 리뷰 보고서

리뷰번호	008	작성자	정제은	작성날짜	2026.06.02
관련규칙	41. 배열이나 컬렉션을 반환하는 메서드에서 널 값 대신 빈 배열이나 컬렉션을 반환하라 63. 과도한 코드와 값을 찾아 제거하라				
중요도	상				
대상코드	Before				
	<pre>package crypto; import java.io.ByteArrayInputStream; import java.io.ByteArrayOutputStream; import java.io.IOException; import java.io.ObjectInputStream; import java.io.ObjectOutputStream; public class SerializeUtil { // 객체 직렬화 (Object -> byte[]) public static byte[] objectToBytes(Object obj) throws IOException { try (ByteArrayOutputStream bos = new ByteArrayOutputStream(); ObjectOutputStream oos = new ObjectOutputStream(bos)) { oos.writeObject(obj); oos.flush(); return bos.toByteArray(); } } // 객체 역직렬화 (byte[] -> Object) public static Object bytesToObject(byte[] data) throws IOException, ClassNotFoundException { try (ByteArrayInputStream bis = new ByteArrayInputStream(data); ObjectInputStream ois = new ObjectInputStream(bis)) { return ois.readObject(); } } }</pre>				
대상코드	After				
	<pre>public class SerializeUtil { // 객체 직렬화 (Object -> byte[]) public static byte[] objectToBytes(Object obj) throws IOException { if (obj == null) { return new byte[0]; } try (ByteArrayOutputStream bos = new ByteArrayOutputStream(); ObjectOutputStream oos = new ObjectOutputStream(bos)) { oos.writeObject(obj); return bos.toByteArray(); } } // 객체 역직렬화 (byte[] -> Object) public static Object bytesToObject(byte[] data) throws IOException, ClassNotFoundException { if (data == null data.length == 0) { return null; } try (ByteArrayInputStream bis = new ByteArrayInputStream(data); ObjectInputStream ois = new ObjectInputStream(bis)) { return ois.readObject(); } } }</pre>				
보안약점/ 보안취약점 이유 및 해결법	41) NullPointerException을 예방하여 obj가 null인지 확인하는 코드를 추가했다. 63) try-with-resources문법을 사용 시 스트림이 닫힐 때 flush()가 자동 호출되므로 중복된 코드를 제거하여 코드의 간결성과 가독성을 높였다.				

코드 리뷰 보고서

리뷰번호	009	작성자	정제은	작성날짜	2026.06.02
관련규칙	01. 민감-데이터의 수명을 제한하라 02. 클라이언트에 암호화되지 않은 민감-데이터를 저장하지 말라 26. 메시드의 결과값에 대해 항상 피드백을 제공하라 41. 배열이나 컬렉션을 반환하는 메서드에서 널 값 대신 빈 배열이나 컬렉션을 반환하라				
중요도	상				
대상코드	Before				
	<pre> System.out.println(); System.out.println("---- 결제 정보 입력 ----"); System.out.print("카드 번호를 입력하세요: "); String cardNumber = sc.next(); System.out.println(); public class PaymentInfo implements Serializable { private static final long serialVersionUID = 1L; private String cardNumber; private int price; public PaymentInfo(String cardNumber, int price) { this.cardNumber = cardNumber; this.price = price; } public String getCardNumber() { return cardNumber; } public int getPrice() { return price; } @Override public String toString() { // 보안 가이드라인 준수; 로그나 디버깅 출력 시 민감한 카드번호 미스형 처리 String maskedCard = (cardNumber != null && cardNumber.length() > 4) ? cardNumber.substring(0, 4) + "----" : "----"; return "카드번호: " + maskedCard + ", 결제금액: " + price; } } </pre>				
대상코드	After				
	<pre> System.out.println(); System.out.println("---- 결제 정보 입력 ----"); System.out.print("카드 번호를 입력하세요: "); char[] inputCardNumber = sc.next().toCharArray(); System.out.println(); public class PaymentInfo implements Serializable { private static final long serialVersionUID = 1L; private char[] cardNumber; private int price; public PaymentInfo(char[] cardNumber, int price) { this.cardNumber = cardNumber != null ? cardNumber.clone() : null; this.price = price; } public char[] getCardNumber() { return cardNumber != null ? cardNumber.clone() : new char[0]; } public int getPrice() { return price; } // 카드번호 표시 지우기 public void clearCardNumber() { if (this.cardNumber != null) { Arrays.fill(this.cardNumber, ' '); // 메모리 공간은 문자 ' '으로 덮어씌움 } } public String toString() { if (cardNumber == null cardNumber.length == 0) { return "카드번호: ----"; } StringBuilder masked = new StringBuilder(); for (int i = 0; i < cardNumber.length; i++) { if (i < 4) { masked.append(cardNumber[i]); // 앞 4자리만 원문 노출 } else { if (cardNumber[i] == ' ') { masked.append(" "); } else { masked.append(" "); } } } return masked.toString(); } } </pre>				
보안약점/ 보안취약점 이유 및 해결법	String객체는 한 번 생성되면 메모리에서 내용을 수정하거나 지울 수 없어서 카드번호(민감 데이터)가 평문에 그대로 남게 된다. 그래서 char[]로 변경하였고, Arrays.fill()을 사용하여 데이터를 즉시 파기하도록 수정하였다. 26) 외부 주소값 조작 방지를 위해 생성자 내에 방어적 복사를 했다. 41)외부 호출 측의 NullPointerException을 막기 위해 null일 경우 빈 배열을 반환하도록 했다.				

코드 리뷰 보고서

리뷰번호	010	작성자	정제은	작성날짜	2026.06.02
관련규칙	01. 민감-데이터의 수명을 제한하라 24. 클래스와 멤버들에 대한 접근성을 최소화하라 33. 일반적인 예외의 타입보다는 사용자-정의 예외를 사용하라 64. 논리적 완벽을 추구하라				
중요도	상				
대상코드	Before				
	<pre>public class CardCompany { private PrivateKey privateKey; public CardCompany(PrivateKey privateKey) { this.privateKey = privateKey; } // 비밀의 민트에서 보낸 2차 전자봉투 개봉 public BaeminToCardData openEnvelope(DigitalEnvelope cardEnvelope) throws InvalidKeyException, NoSuchAlgorithmException, NoSuchPaddingException, IllegalBlockSizeException, BadPaddingException, ClassNotFoundException, IOException { byte[] decrypted = EnvelopeUtil.openEnvelope(cardEnvelope, privateKey); return (BaeminToCardData) SerializableUtil.bytesToObject(decrypted); } // 1차 전자봉투 개봉 public CardData openCardEnvelope(BaeminToCardData cardData) throws InvalidKeyException, NoSuchAlgorithmException, NoSuchPaddingException, IllegalBlockSizeException, BadPaddingException, ClassNotFoundException, IOException { DigitalEnvelope cardEnvelope = cardData.getCardEnvelope(); byte[] decrypted = EnvelopeUtil.openEnvelope(cardEnvelope, privateKey); return (CardData) SerializableUtil.bytesToObject(decrypted); } // 결제 정보 검증 public boolean verifyPayment(CardData cardData, PublicKey userPublicKey) throws IOException, NoSuchAlgorithmException, InvalidKeyException, SignatureException { // 결제 정보 해시 생성 PaymentInfo paymentInfo = cardData.getPaymentInfo(); byte[] paymentInfoBytes = SerializableUtil.objectToBytes(paymentInfo); byte[] paymentInfoHash = HashUtil.hash(paymentInfoBytes); // 주문 정보 해시, 결제 정보 해시 결합 byte[] orderInfoHash = cardData.getOrderInfoHash(); byte[] combined = HashUtil.combineHash(orderInfoHash, paymentInfoHash); // 최종 해시 값 생성 byte[] combinedHash = HashUtil.hash(combined); // 결제 정보 검증 byte[] digitalSignature = cardData.getDigitalSignatures(); return SignatureUtil.digitalsignVerify(combinedHash, digitalSignature, userPublicKey); } }</pre>				
대상코드	After				
	<pre>// 사용자 정의 예외 생성 class CardCompanyException extends Exception { private static final long serialVersionUID = 1L; public CardCompanyException(String message, Throwable cause) { super(message, cause); } } public class CardCompany { private final PrivateKey privateKey; public CardCompany(PrivateKey privateKey) { this.privateKey = privateKey; } // 비밀의 민트에서 보낸 2차 전자봉투 개봉 public BaeminToCardData openCardEnvelope(DigitalEnvelope cardEnvelope) throws CardCompanyException { byte[] decrypted = null; try { decrypted = EnvelopeUtil.openEnvelope(cardEnvelope, privateKey); return (BaeminToCardData) SerializableUtil.bytesToObject(decrypted); } catch (Exception e) { throw new CardCompanyException("2차 전자봉투 개봉 중 오류가 발생했습니다.", e); } finally { if (decrypted != null) { Arrays.fill(decrypted, (byte) 0); } } } // 1차 전자봉투 개봉 public CardData openCardEnvelope(BaeminToCardData cardData) throws CardCompanyException { byte[] decrypted = null; try { DigitalEnvelope cardEnvelope = cardData.getCardEnvelope(); decrypted = EnvelopeUtil.openEnvelope(cardEnvelope, privateKey); return (CardData) SerializableUtil.bytesToObject(decrypted); } catch (Exception e) { throw new CardCompanyException("1차 결제 전자봉투 개봉 중 오류가 발생했습니다.", e); } finally { if (decrypted != null) { Arrays.fill(decrypted, (byte) 0); } } } // 결제 정보 검증 public boolean verifyPayment(CardData cardData, PublicKey userPublicKey) throws CardCompanyException { try { // 결제 정보 해시 생성 PaymentInfo paymentInfo = cardData.getPaymentInfo(); paymentInfoBytes = SerializableUtil.objectToBytes(paymentInfo); byte[] salt = cardData.getPaymentInfoSalt(); paymentInfoHash = HashUtil.hashSalt(paymentInfoBytes, salt); // 주문 정보 해시, 결제 정보 해시 결합 byte[] orderInfoHash = cardData.getOrderInfoHash(); combined = HashUtil.combineHash(orderInfoHash, paymentInfoHash); // 최종 해시 값 생성 combinedHash = HashUtil.hash(combined); // 결제 정보 검증 byte[] digitalSignature = cardData.getDigitalSignatures(); return SignatureUtil.digitalsignVerify(combinedHash, digitalSignature, userPublicKey); } catch (Exception e) { throw new CardCompanyException("결제 정보 인증서명 검증 중 오류가 발생했습니다.", e); } finally { // 메모리에 남아 있는 민감 데이터 제거 if (paymentInfoBytes != null) { Arrays.fill(paymentInfoBytes, (byte) 0); } if (paymentInfoHash != null) { Arrays.fill(paymentInfoHash, (byte) 0); } if (combined != null) { Arrays.fill(combined, (byte) 0); } if (combinedHash != null) { Arrays.fill(combinedHash, (byte) 0); } } if (cardData != null && cardData.getPaymentInfo() != null) { cardData.getPaymentInfo().clearCardNumber(); } } }</pre>				
보안약점/ 보안취약점 이유 및 해결법	01) 전자봉투를 열어서 나온 평문 바이트 배열, 이중 서명 검증을 위한 해시 버퍼들이 연산이 끝난 후에도 남아있게 된다. 그래서 finally블록에서 전부 0으로 채웠다. 24) privateKey멤버 변수에 final을 넣어서 객체 생성 후 메모리 참조가 악의적으로 변경되지 않도록 수정하였다. 33) throws 예외들이 날것으로 있어서 시스템 구조를 파악당할 수 있다. 그래서 CardCompanyException이라는 사용자 정의 예외 클래스를 생성하여 내부 캡슐화가 되어 외부 유출 가능성이 낮아졌다. 64) 어떤 예외가 터져도 방어할 수 있게 수정하였다.				

코드 리뷰 보고서

리뷰번호	011	작성자	정제은	작성날짜	2026.06.02
관련규칙	01. 민감-데이터의 수명을 제한하라 26. 메시드의 결과값에 대해 항상 피드백을 제공하라 33. 일반적인 예외의 타입보다는 사용자-정의 예외를 사용하라 64. 논리적 완벽을 추구하라				
중요도	상				
대상코드	Before				
	<pre>public class Store { private PrivateKey privateKey; public Store (PrivateKey privateKey) { this.privateKey = privateKey; } // 배열의 인력이 보낸 전자봉투 개를 public BaeminInfoStoreData openEnvelope(DigitalEnvelope storeEnvelope) throws InvalidKeyException, NoSuchAlgorithmException, NoSuchPaddingException, IllegalBlockSizeException, BadPaddingException, ClassNotFoundException, IOException { byte[] decrypted = EnvelopeUtil.openEnvelope(storeEnvelope, privateKey); return (BaeminInfoStoreData) SerializeUtil.bytesToObject(decrypted); } // 주문 정보 검증 public boolean verifyOrder(BaeminInfoStoreData storeData, PublicKey userPublicKey) throws IOException, NoSuchAlgorithmException, InvalidKeyException, SignatureException { // 주문 정보 해시 생성 OrderInfo orderInfo = storeData.getOrderInfo(); byte[] orderInfoBytes = SerializeUtil.objectToBytes(orderInfo); byte[] orderInfoHash = HashUtil.hash(orderInfoBytes); // 주문 정보 해시, 결제 정보 해시 결합 byte[] paymentInfoHash = storeData.getPaymentInfoHash(); byte[] combined = HashUtil.combineHash(orderInfoHash, paymentInfoHash); // 최종 해시 값 해시 byte[] combinedHash = HashUtil.hash(combined); // 이종서명 검증 byte[] digitalSignature = storeData.getDigitalSignature(); return SignatureUtil.digitalSignVerify(combinedHash, digitalSignature, userPublicKey); } }</pre>				
	After				
	<pre>class StoreCryptoException extends Exception { private static final long serialVersionUID = 1L; public StoreCryptoException(String message, Throwable cause) { super(message, cause); } } public class Store { private final PrivateKey privateKey; public Store (PrivateKey privateKey) { this.privateKey = privateKey; } // 배열의 인력이 보낸 전자봉투 개를 public BaeminInfoStoreData openEnvelope(DigitalEnvelope storeEnvelope) throws StoreCryptoException{ byte[] decrypted = null; try { decrypted = EnvelopeUtil.openEnvelope(storeEnvelope, privateKey); return (BaeminInfoStoreData) SerializeUtil.bytesToObject(decrypted); } catch (Exception e) { throw new StoreCryptoException("가맹점 전자봉투 개봉 중 오류가 발생했습니다.", e); } finally { if (decrypted != null) { Arrays.fill(decrypted, (byte) 0); } } } // 주문 정보 검증 public boolean verifyOrder(BaeminInfoStoreData storeData, PublicKey userPublicKey) throws StoreCryptoException { byte[] orderInfoBytes = null; byte[] orderInfoHash = null; byte[] combined = null; byte[] combinedHash = null; try { // 주문 정보 해시 생성 OrderInfo orderInfo = storeData.getOrderInfo(); orderInfoBytes = SerializeUtil.objectToBytes(orderInfo); byte[] orderSalt = storeData.getOrderInfoSalt(); orderInfoHash = HashUtil.hashSalt(orderInfoBytes, orderSalt); // 주문 정보 해시, 결제 정보 해시 결합 byte[] paymentInfoHash = storeData.getPaymentInfoHash(); combined = HashUtil.combineHash(orderInfoHash, paymentInfoHash); // 최종 결합 해시 생성 combinedHash = HashUtil.hash(combined); byte[] digitalSignature = storeData.getDigitalSignature(); return SignatureUtil.digitalSignVerify(combinedHash, digitalSignature, userPublicKey); } catch (Exception e) { throw new StoreCryptoException("주문 정보 이종서명 검증 중 오류가 발생했습니다.", e); } finally { if (orderInfoBytes != null) Arrays.fill(orderInfoBytes, (byte) 0); if (orderInfoHash != null) Arrays.fill(orderInfoHash, (byte) 0); if (combined != null) Arrays.fill(combined, (byte) 0); if (combinedHash != null) Arrays.fill(combinedHash, (byte) 0); } } }</pre>				
보안약점/ 보안취약점 이유 및 해결법	01) 민감 데이터를 Arrays.fill()을 사용하여 데이터 바로 제거 26) 연산 도중 예외 발생 시 이를 침묵시키거나 모호한 성공/실패값을 주게 되면 상위 시스템의 오작동이 생길 수 있어 정확한 예외 피드백을 주어 안전하게 취소될 수 있도록 변경 33) 예외 발생 시 날것 그대로 노출될 수 있어서 StoreCryptoException을 만들어서 내부에 들어가도록 수정하였다. 64) 모든 예외를 방어할 수 있도록 수정하였다.				

코드 리뷰 보고서

리뷰번호	012	작성자	정제은	작성날짜	2026.06.02
관련규칙	01. 민감-데이터의 수명을 제한하라 26. 메서드의 결과값에 대해 항상 피드백을 제공하라 33. 일반적인 예외의 타입보다는 사용자-정의 예외를 사용하라 64. 논리적 완벽을 추구하라				
중요도	상				
대상코드	Before				
	After				
보안약점/ 보안취약점 이유 및 해결법	01) 민감 데이터를 Arrays.fill()을 사용하여 데이터 바로 제거 26) 연산 도중 예외 발생 시 이를 침묵시키거나 모호한 성공/실패값을 주게 되면 상위 시스템의 오작동이 생길 수 있어 정확한 예외 피드백을 주어 안전하게 취소될 수 있도록 변경 33) 예외 발생 시 날것 그대로 노출될 수 있어서 UserCryptoException을 만들어서 내부에 들어가도록 수정하였다. 64). 모든 예외를 방어할 수 있도록 수정하였다.				

코드 리뷰 보고서

리뷰번호	013	작성자	정제은	작성날짜	2026.06.02
관련규칙	01. 민감-데이터의 수명을 제한하라 24. 클래스와 멤버들에 대한 접근성을 최소화하라 33. 일반적인 예외의 타입보다는 사용자-정의 예외를 사용하라				
중요도	상				
대상코드	Before				
	<pre>public class Baemin { private PrivateKey privateKey; public Baemin(PrivateKey privateKey) { this.privateKey = privateKey; } // 사용자 보낸 전자봉투 개봉 public UserToBaeminData openEnvelope(DigitalEnvelope baeminEnvelope) throws IOException, NoSuchAlgorithmException, NoSuchPaddingException, IllegalBlockSizeException, BadPaddingException, ClassNotFoundException, IOException { byte[] decrypted = EnvelopeUtil.openEnvelope(baeminEnvelope, privateKey); UserToBaeminData baeminData = (UserToBaeminData) bytesToObject(decrypted); return baeminData; } // 가맹점 전달 데이터 생성 public BaeminStoreData generateStoreData(UserToBaeminData data) { BaeminStoreData storeData = new BaeminStoreData(data.getOrderInfo(), data.getPaymentInfoHash(), data.getDigitalSignature()); return storeData; } // 가맹점 전달 전자봉투 생성 public DigitalEnvelope createStoreEnvelope(BaeminStoreData storeData, PublicKey storePublicKey) throws IOException, NoSuchAlgorithmException, NoSuchPaddingException, IllegalBlockSizeException, BadPaddingException { byte[] storeDataBytes = objectToBytes(storeData); DigitalEnvelope storeEnvelope = EnvelopeUtil.sealEnvelope(storeDataBytes, storePublicKey); return storeEnvelope; } // 카드사 전달 데이터 생성 public BaeminCardData generateCardData(UserToBaeminData data) { BaeminCardData cardData = new BaeminCardData(data.getCardEnvelope()); return cardData; } // 카드사 전달 전자봉투 생성 public DigitalEnvelope createCardEnvelope(BaeminCardData cardData, PublicKey cardPublicKey) throws IOException, NoSuchAlgorithmException, NoSuchPaddingException, IllegalBlockSizeException, BadPaddingException { byte[] cardDataBytes = objectToBytes(cardData); DigitalEnvelope cardEnvelope = EnvelopeUtil.sealEnvelope(cardDataBytes, cardPublicKey); return cardEnvelope; } public Object bytesToObject(byte[] data) throws IOException, ClassNotFoundException { ByteArrayInputStream bis = new ByteArrayInputStream(data); ObjectInputStream ois = new ObjectInputStream(bis); Object object = ois.readObject(); return object; } }</pre>				
대상코드	After				
	<pre>// 사용자 정의 예외 생성 class BaeminCryptoException extends Exception { private static final long serialVersionUID = 1L; public BaeminCryptoException(String message, Throwable cause) { super(message, cause); } } public class Baemin { private final PrivateKey privateKey; public Baemin(PrivateKey privateKey) { this.privateKey = privateKey; } // 사용자 보낸 전자봉투 개봉 public UserToBaeminData openEnvelope(DigitalEnvelope baeminEnvelope) throws BaeminCryptoException { byte[] decrypted = null; try { decrypted = EnvelopeUtil.openEnvelope(baeminEnvelope, privateKey); return (UserToBaeminData) SerializableUtil.bytesToObject(decrypted); } catch (Exception e) { throw new BaeminCryptoException("해당데이터를 수신 전자봉투 해독 중 오류가 발생했습니다.", e); } finally { if (decrypted != null) { Arrays.fill(decrypted, (byte) 0); } } } // 가맹점 전달 데이터 생성 public BaeminStoreData generateStoreData(UserToBaeminData data) { return new BaeminStoreData(data.getOrderInfo(), data.getPaymentInfoHash(), data.getDigitalSignature(), data.getOrderInfoSalt()); } // 가맹점 전달 전자봉투 생성 public DigitalEnvelope createStoreEnvelope(BaeminStoreData storeData, PublicKey storePublicKey) throws BaeminCryptoException { byte[] storeDataBytes = null; try { storeDataBytes = SerializableUtil.objectToBytes(storeData); return EnvelopeUtil.sealEnvelope(storeDataBytes, storePublicKey); } catch (Exception e) { throw new BaeminCryptoException("가맹점용 전자봉투 생성 도중 오류가 발생했습니다.", e); } finally { if (storeDataBytes != null) { Arrays.fill(storeDataBytes, (byte) 0); } } } // 카드사 전달 데이터 생성 public BaeminCardData generateCardData(UserToBaeminData data) { return new BaeminCardData(data.getCardEnvelope()); } // 카드사 전달 전자봉투 생성 public DigitalEnvelope createCardEnvelope(BaeminCardData cardData, PublicKey cardPublicKey) throws BaeminCryptoException { byte[] cardDataBytes = null; try { cardDataBytes = SerializableUtil.objectToBytes(cardData); return EnvelopeUtil.sealEnvelope(cardDataBytes, cardPublicKey); } catch (Exception e) { throw new BaeminCryptoException("카드사용 전자봉투 생성 도중 오류가 발생했습니다.", e); } finally { if (cardDataBytes != null) { Arrays.fill(cardDataBytes, (byte) 0); } } } }</pre>				
보안약점/ 보안취약점 이유 및 해결법	01) 민감 데이터를 Arrays.fill()을 사용하여 데이터 바로 제거 24) privateKey멤버 변수에 final을 넣어서 객체 생성 후 메모리 참조가 악의적으로 변경되지 않도록 수정하였다. 33) 예외 발생 시 날것 그대로 노출될 수 있어서 UserCryptoException을 만들어서 내부에 들어가도록 수정하였다.				